01 / Team status and completion

Revised two-day integration plan | 20-22 October 2026 | Asia/Shanghai

Target: connect real P1, P3, P4 and P5 modules through P2 and demonstrate the supported flows in the frontend. Updated assignments: P1 leads agent-to-backend-to-chat integration with P2; P3 owns OTP/MFA implementation; P4 retains security-policy decisions and review.

Person / estimate	Completed in reviewed code	Main remaining work
P1 / Claine ~80%	Gemini + offline provider; five intents; transfer, spending and cancellation memory; result correlation; report summaries; tests and rehearsal.	Lead real agent/state/result connection to P2; align frontend chat/approval presentation; prove supported conversations through HTTP/UI.
P2 / Marvel ~65%	DB and read APIs; action/digest/approval flow; transfer executor and simulated cancellation; React dashboard, confirmation/MFA screens; Docker scaffolding.	Coordinate real module dispatch, ownership and action mapping; fix metadata/state gaps; persistent cancellation state; packaging and app-level verification.
P3 / finance + MFA ~70%	Exact Decimal analytics; category/anomaly rules; local-month logic; diagnostics, examples and tests. A basic fixed-code MFA flow already exists in P2 code.	Connect finance with P2; newly own OTP lifecycle, expiry/attempts and verify flow with P4 review. Existing MFA code is a starting point, not evidence this new assignment is complete.
P4 / security ~85%	Pure fail-closed policy; strict argument/account checks; Green/Yellow/Red and daily threshold rules; lock configuration; extensive tests and security notes.	Connect real policy, review execution-time checks and P3 MFA; verify rejection, expiry, replay and concurrent execution behaviour.
P5 / subscriptions ~80%	Recurring detector; cadence/confidence/evidence; duplicate handling; cancellation eligibility; CLI, tests and backend-mapping proposal.	Wire real detection/eligibility; agree candidate-to-record mapping; verify support/status and simulated cancelled state with P2.

How to read percentages: approximate planning estimates for each person's current full role, including integration and proof. Allow roughly +/-10 percentage points. They are not measured test coverage, hours worked or a ranking. P3's expanded role includes new MFA responsibility; finance alone is substantially more complete.

Fresh baseline: 389 passed, 1 failed, 1 existing SDK warning. The failing case is backend/tests/test_p1_integration.py::test_real_agent_balance_inquiry_read_loop. A large module-test pass count does not prove end-to-end integration.

02 / Review basis and priorities

Latest remote checked before authoring | do these blockers before adding features

Fetched origin successfully. origin/main = 041c84e, matching the reviewed working tree. origin/feature/app = 82eb828 differs from main only by the two trial comments in adapters.py. Unpushed work on other machines is not visible. This replaces the earlier plan's assignment of OTP implementation to P2.

Priority	Current code evidence	Owner / next action
1 - state / results	Adapter calls process_message and chat stores clarification text. Current P1 requires returned ConversationState with original pending proposals; read correlation fails.	P1 + P2: use process_turn or a wrapper; save every returned state; pass exact result envelope.
1 - real bridges	Agent global instance forces use_mock=True. Finance, policy and detector non-mock branches still return their mock implementations.	P2 coordinates: wire real calls with module owners; make configuration control actual instances.
1 - read dispatch	chat.py uses TransactionModel and datetime without imports. Some account lookup queries are not owner-scoped.	P1 + P2: fix read path; P3/P5 verify transaction scope/history.
2 - clean action data	Chat inserts action_id/digest/tier into strict tool arguments. Stored payloads are cleaner, but returned proposals are still polluted.	P1 + P2: agree metadata transport with UI; P4 validates pure canonical args.
2 - OTP / execution	Fixed 123456 and attempt counting exist. Verify path has no expiry recheck; execution lock/policy review and real audit correlation remain.	P3 implements challenge/verify fixes; P4 reviews controls; P2 owns balance/ledger executor.
2 - subscription truth	Detector returns candidates, not verified status; cancellation executor currently writes an audit entry without a stored cancelled state.	P5 + P2: trusted record mapping and persistent demo state; P1 must not upgrade a candidate to active.

Keep the sprint bounded

No new providers, general conversation memory, arbitrary corrections, real bank transfers or mandatory SMS delivery. Keep five supported intents and a guided connected sequence. Detailed diagnostics, single-command orchestration and visual redesign wait until all acceptance gates pass.

Packaging blocker: Docker backend currently copies backend/ and contracts/ only. Before container rehearsal, include the four real module packages and their dependencies, seed inputs, and selected modes. Frontend build/runtime must be checked; Docker files existing is not a deployment pass.

03 / Day 1: integration schedule

Approximately 7-8 focused hours each | relative blocks from the team kickoff

Time	Shared checkpoint	Parallel work / deliverable
0:00-0:45	All five: G0 agreement	Same base commit and baseline. Agree ownership, user/conversation state keys, result envelope, action metadata/UI representation and OTP simulation scope.
0:45-2:45	P1 + P2: agent / memory	P1 leads assigned adapter/chat/session integration; P2 supplies DB context and reviews. P3 implements OTP lifecycle tests in agreed MFA files. P4 prepares real policy bridge/vectors. P5 prepares detector-to-record mapping and expected outputs.
2:45-4:15	P2 + P3/P5: reads	P2 connects finance and detector dispatch using prepared examples. P1 checks frontend chat and result formatting. P3/P5 verify selected account and history. P4 checks Green authorization and locked reads.
4:15-5:15	P2 + P4: policy bridge	Real evaluate_action on canonical proposals; daily boundaries and source-account checks. P3 tests MFA confirm/verify with P4 and prepares endpoint/UI handoff.
5:15-6:15	P1 + P2: UI slice	One real balance and spending clarification round trip visible in browser. Align action metadata representation; do not silently modify frozen DTOs. Others finish module regression checks.
6:15-7:00	All five: G1 gate	Merge narrow reviewed PRs. Run read/state/policy tests with real modules. Record commit, modes, dataset, failing requests and owners. Use remaining hour for blockers, not new scope.

G1 must be demonstrated, not assumed

• HTTP balance reflects the database, not a fixed number; spending clarification survives invalid answers; list_subscriptions uses the real detector and reaches P1 as a matched report.

• Two users/conversations are isolated; never mind clears only an unfinished draft. P4 policy actually runs. P3 OTP code and tests are ready for Day 2 integration, with clear unresolved controls.

Avoid shared-file collisions

P2 coordinates shared backend files. At kickoff, explicitly delegate P1's adapter/chat/session/frontend scope and P3's MFA scope. P1 owns the first adapter edit window, then hands it back to P2 for P3/P4/P5 bridges. No simultaneous edits to the same adapter or approval file. Use personal branches and short PRs; do not have all five push feature/app.

04 / P1: connect the conversation

Lead: Claine | partners: P2 for API/UI, P3 for reports, P5 for verified context

Day 1 implementation order

• 1. Agent bridge: use process_turn(chat, catalog, tool_results) so the backend gets both public turn.result and private turn.state. Keep process_message and shared AgentResult unchanged. Real P1 service + AGENT_PROVIDER=mock is the deterministic integration mode.

• 2. Trusted context: one selected owned account, lock status, verified recipients and Subscription records; same state snapshot on every catalog entry. No hardcoded fallback account or cross-user fixture defaults in real mode.

• 3. Memory: store by authenticated user + conversation, serialize updates, and save state after every turn. Do not mix legacy clarification_request with active structured state. Never mind discards drafts but preserves issued proposals awaiting results.

• 4. Read loop: preserve original proposals in pending_tool_requests. P2 performs authorized lookup, supplies matching conversation/proposal/tool/arguments plus status/data, then saves the newly consumed state. One result per continuation.

• 5. Frontend chat: preserve the correct conversation across turns, render multiline report replies clearly, display pending/failed/completed distinctly and refresh affected views. P1 focuses on chat integration; P2 remains app owner. P3 owns OTP mechanics, not P1.

First-hour interface decisions

ChatOutput currently lacks needs_clarification and a clean action-metadata slot. P1 + P2 must choose an explicit mapping or a coordinated versioned contract update with UI and tests. Keep canonical tool arguments pure. Do not silently add fields or place server metadata where strict validators reject it.

Server action_id is different from P1 proposal_id. P2 persists the mapping with user/conversation and returns trusted ExecutionResult to P1. P1 must not claim completion from a proposal, a confirmation click or a pending MFA result.

Acceptance owned by P1

Conversation	Pass condition
Analyze my spending -> invalid month -> 2026-10	Same draft retained; correct account/month proposal; matching report summary only after trusted result.
Transfer to Bob -> 0 -> 500	Invalid amount does not erase recipient; 500.00 proposal uses current verified recipient ID.
List subscriptions -> Cancel my subscription -> name	Listing alone grants no authority; current verified subscription determines cancellation ID/support.
Wrong result / never mind / another user	Wrong correlation rejected; draft abandonment does not revoke issued actions; state never crosses owners.

Day 2: prove these through HTTP and browser, then a few Gemini initial requests. If Gemini fails, report provider failure separately; keep the offline module-integrated path runnable.

05 / P2 + P3: app and OTP

P3 owns OTP/MFA implementation | P2 owns app infrastructure and money execution

P2 / Marvel - integration coordinator

• Wire real finance, policy and detector bridges; make mock flags honest. Use selected-account authorized queries and typed shared DTOs. Keep original action arguments unchanged, preserve source/recipient IDs and own action storage, digest and ledger transaction.

• Coordinate with P1 on state/read loops/UI metadata; with P3 on confirm/verify and MFA screens; with P4 on execution-time checks; with P5 on stored subscription state. No participant should need to rewrite another's calculation logic.

• Persist proposal/action/user/conversation linkage, actual audit IDs and cancellation status in the demo store. Unknown tools fail closed. Confirmation dispatch must whitelist transfer vs cancellation, not send arbitrary tools to the transfer executor.

• Fix clean startup/schema changes and container packaging. Copy agent, finance, security and subscriptions into backend image; install provider dependencies; inject environment modes/key securely. Build frontend against supported toolchain and confirm API proxy routes.

P3 - finance support plus the new MFA assignment

• Start from existing code: approvals.py already issues fixed 123456, exposes PENDING_MFA and counts bad attempts. Audit what works before replacing it. P3 owns the MFA endpoint logic and focused tests in files agreed with P2; do not rewrite P2's executor.

• Define the challenge: bind it to the owned backend action and its current state. Use a clear issue/expiry time, failed-attempt limit and consumed state. Check expiry again on verify; reject wrong user, expired, failed or already-consumed challenge without executing again.

• Enforce flow: confirmation of a Red action leads to PENDING_MFA. Only valid verification permits P2's executor. P4 specifies when MFA is required and reviews lock/current-policy checks. Handle valid retries/idempotent completion without a second debit.

• Deliver to UI: P3 and P2 agree response/error states; P3 supplies code-entry behaviour, wrong-code/expiry messages and retry rules. Keep P1 out of code generation/verification. Label fixed test OTP as simulated; no SMS/email is currently sent.

• Finance support: P3 verifies selected-account inputs, requested month plus prior three local months, Decimal values and raw credits/refunds/duplicates. Do not rebuild analytics while taking on MFA.

Evidence by Day 2 midday

Red transfer: no debit after chat or confirm, one debit only after valid MFA. Wrong code, third failed attempt, expiry, lock change and wrong-user tests reject safely. A duplicate confirmation/verification cannot debit twice. Test with P2's executor and P4's actual policy, not only isolated MFA functions.

Choose simulated OTP for this sprint unless organizer rules require actual delivery. If real delivery is required, schedule it explicitly rather than quietly assuming a fixed code satisfies the competition.

06 / P4 + P5: policy and records

Module owners pair with P2 | P4 reviews P3 MFA, not vice versa

P4 - real policy integration and review

• Connect security.policy.evaluate_action; eliminate hidden mock fallback. Verify shared arguments before metadata, current authenticated BankingContext, source/account matching, explicit rejection, and default lock behaviour (reads allowed, mutations blocked).

• Daily cumulative boundaries: 0 + 1000 => Yellow; 0 + 1000.01 => Red; 800 + 300 => Red. Include zero/negative/precision/currency errors, tampered digest, wrong user and unknown tool.

• Review P3's MFA challenge binding, expiry, bad-attempt/consumption logic. Review with P2 that lock and policy are rechecked at execution. A correct OTP alone cannot override expiry or a newly forbidden action.

• Provide simultaneous/repeated request checks: one atomic action transition, one ledger debit, consistent audit IDs. Existing sequential replay success is not evidence of concurrent idempotency.

P5 - detector, eligibility and candidate-to-record mapping

• Use real detect_subscriptions(transactions) with sufficient same-account history. Verify candidate ID, median estimated amount, cadence, confidence and evidence IDs. Annual detection needs a longer history than the finance three-month baseline.

• Raw detector cancellation_supported=False is intentional. P2 enriches it from a stored Subscription record scoped to the user. Candidate IDs and stored IDs need an explicit verified mapping; same name is not sufficient if ambiguous.

• Current P2/P5 notes agree confidence >=0.70 for an active classification and lower scores as possible. Confirm the meaning: this heuristic must not replace trusted active status, cancellation support or authorization. Unknown records remain candidates.

• Call assess_cancellation on the supported/enriched candidate, then separately check stored active/cancelled status, since SubscriptionCandidate has no status field. P5 supplies semantics; P2 persists and executes the controlled demo cancellation.

• Acceptance: list -> clarify -> supported verified name creates the right proposal. Unknown/unsupported/inactive/duplicate merchant names stay blocked. After simulated cancellation, listing/context reflects cancelled state; repeated cancellation has a defined safe outcome.

Finance evidence P3 supplies while working on MFA

Shared October report: 1843.00, with six exact category totals and no anomalies. P3 September dataset: 9254.70 and seven full flags. Use those datasets deliberately; backend test seed has different amounts. Confirm Asia/Shanghai month edges and no refund-netting claims. Diagnostic notes remain outside SpendingReport for this sprint.

Boundary between modules

P1 explains and proposes. P3 calculates and now implements MFA mechanics. P4 decides policy. P5 detects and assesses eligibility. P2 orchestrates, stores identity/state and performs approved demo mutations. A stub or isolated unit pass must not be described as complete cross-module integration.

07 / Day 2: app-level proof

Finish the connected demo before cosmetic improvements | keep an hour for blockers

Time	Owners	Output / checkpoint
0:00-1:00	All five; P2 coordinates	Start from Day 1 merged commit. Re-run reads/state and fix G1 blockers. P3 verifies MFA patch is on the same base; P1 checks browser chat.
1:00-3:00	P3 + P2 + P4	Integrate MFA and actual policy into confirmation/execution. P5 + P2 finish simulated cancelled-state persistence. P1 checks returned outcomes and UI status. G2 mutation controls pass.
3:00-4:00	P1 leads; P2 + P5; P4 checks	Guided app conversation: list -> verified subscription selection -> cancellation confirmation -> trusted simulated outcome -> explicit transfer request -> separate confirmation/MFA if policy requires.
4:00-5:00	P1 + P2; P3 verifies OTP UI	Browser chat, clarification, action review, MFA input and errors all match endpoints. Refresh balances, ledger, subscriptions and audit on completion. No raw stale metadata mismatch.
5:00-6:00	All five	Run combined suites and HTTP/browser acceptance matrix. Confirm real module modes, clean seed and current commit. Small Gemini smoke check only after deterministic path passes.
6:00-7:00	All five; P2 runs app	Clean startup/build test and rehearsal. Freeze demo candidate, record known limits, assign final blockers. Reserve remaining hour for one focused repair and rerun.

Cross-scenario: keep permissions separate

Use a guided sequence of explicit requests; current P1 deliberately rejects a dependent compound command. Do not promise one-command planning. Cancelling a 39 CNY monthly subscription avoids a possible future charge; it does not add 39 CNY to today's account. A later transfer needs its own amount, verified recipient, available balance and approval. Cancellation confirmation never authorizes transfer.

Parallel work while the app is being fixed

P1: conversation/result/UI tests and demo wording. P3: MFA boundary tests and finance-output fidelity. P4: negative, expiry, policy and concurrency checks. P5: detector mapping and cancelled-state consistency. P2: shared wiring, packaging and startup. Every defect report contains request, actual response, expected response, commit and owner.

G3 completion: all required flows run on one merged commit, real modules are selected, remaining limitations are explicit, and another teammate can start and rehearse the app without the author walking through manual patches.

08 / Acceptance and delivery

Check outcomes rather than hours spent | percentages are not release criteria

Case	Lead / verifier	Required evidence
Real-mode selection	P2 / all owners	Real P1/P3/P4/P5 calls are observable; no silent mock bridge. Mock LLM interpretation is allowed in deterministic tests.
Conversation + reads	P1 / P2	Database balance, spending-month memory, subscription listing; wrong result rejected; never mind and two-user isolation pass through HTTP.
Finance + records	P3/P5 / P1,P2	Exact scoped reports, local-month boundaries and evidence IDs; verified support/status needed before cancellation.
MFA and policy	P3 / P4,P2	No execution before required confirmation/MFA; bad code, expiry, lock change and foreign user blocked; one debit across retries/concurrent requests.
Cancellation + sequence	P5/P2 / P1,P4	Simulated cancellation updates stored state; independent later transfer has separate approval and never credits projected savings.
Frontend + startup	P1/P2 / team	Chat, review, OTP/error views and refreshed data match API; build/start from clean seed; real module packages exist inside backend image.

Reproduce the code-review baseline

```
$env:PYTHONDONTWRITEBYTECODE = '1'
$env:PYTHONPATH = (Get-Location).Path + ';backend'
$env:DATABASE_URL = 'sqlite:///:memory:'
agent/.venv/Scripts/python.exe -m pytest -p no:cacheprovider
tests/test_contracts.py agent/tests backend/tests finance/tests
subscriptions/tests security/tests -q --tb=short
```

Join the last three displayed lines into one pytest command. Baseline on 20 Oct: 389 passed / 1 failed, plus one existing SDK Pydantic warning. Frontend build, live UI, Docker startup and concurrent execution were not validated by this Python run; those are explicit sprint acceptance tasks.

If behind schedule

Protect real module wiring, one grounded spending flow, one verified simulated cancellation and one controlled transfer/MFA flow. Cut styling, extra charts, diagnostics, automatic dependent planning and real OTP delivery unless required by rules. Do not bypass policy or label a stub a completed integration.

End-of-sprint handoff

Record one commit, module/provider modes, startup/seed commands, test results, three short demo conversations, known limitations and owners. Do not claim production banking/security readiness. Review covered current module implementations, contracts, backend routes/adapters/state/execution, frontend API/components, tests and Docker configuration; main was fetched before authoring.